

tf.TensorArraySpec Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/TensorArraySpec

Type specification for a tf.TensorArray.

Function Signatures

```
tf.TensorArraySpec( element_shape=None,  
dtype=tf.dtypes.float32, dynamic_size=False, infer_shape=True  
)
```

Code Examples

```
tf.TensorArraySpec(  
element_shape=None,  
dtype=tf.dtypes.float32,  
dynamic_size=False,  
infer_shape=True  
)
```

```
tf.TensorArraySpec(  
    element_shape=None,  
    dtype=tf.dtypes.float32,  
    dynamic_size=False,  
    infer_shape=True  
)
```

```
tf.dtypes.float32
```

```
experimental_as_proto() -> struct_pb2.TypeSpecProto
```

```
experimental_as_proto() -> struct_pb2.TypeSpecProto
```

```
@classmethod  
experimental_from_proto(  
    proto: struct_pb2.TypeSpecProto  
) -> 'TypeSpec'
```

```
@classmethod
```

```
experimental_from_proto(  
    proto: struct_pb2.TypeSpecProto  
) -> 'TypeSpec'
```

Documentation

Type specification for a `tf.TensorArray`.

Inherits From: `TypeSpec`, `TraceType`

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.TensorArraySpec`

Attributes

Methods

experimental_as_proto

[View source](#)

Returns a proto representation of the TypeSpec instance.

Do NOT override for custom non-TF types.

experimental_from_proto

[View source](#)

Returns a TypeSpec instance based on the serialized proto.

Do NOT override for custom non-TF types.

experimental_type_proto

[View source](#)

Returns the type of proto associated with TypeSpec serialization.

Do NOT override for custom non-TF types.

from_value

[View source](#)

is_compatible_with

[View source](#)

Returns true if spec_or_value is compatible with this TypeSpec.

Prefer using "is_subtype_of" and "most_specific_common_supertype" wherever possible.

is_subtype_of

[View source](#)

Returns True if self is a subtype of other.

Implements the `tf.types.experimental.func.TraceType` interface.

If not overridden by a subclass, the default behavior is to assume the TypeSpec is covariant upon attributes that implement TraceType and invariant upon rest of the attributes as well as the structure and type of the TypeSpec.

most_specific_common_supertype

[View source](#)

Returns the most specific supertype of self and others.

Returns None if a supertype does not exist.

most_specific_compatible_type

[View source](#)

Returns the most specific TypeSpec compatible with self and other. (deprecated)

Deprecated. Please use `most_specific_common_supertype` instead.

Do not override this function.

__eq__

[View source](#)

Return self==value.

__ne__

[View source](#)

Return self!=value.